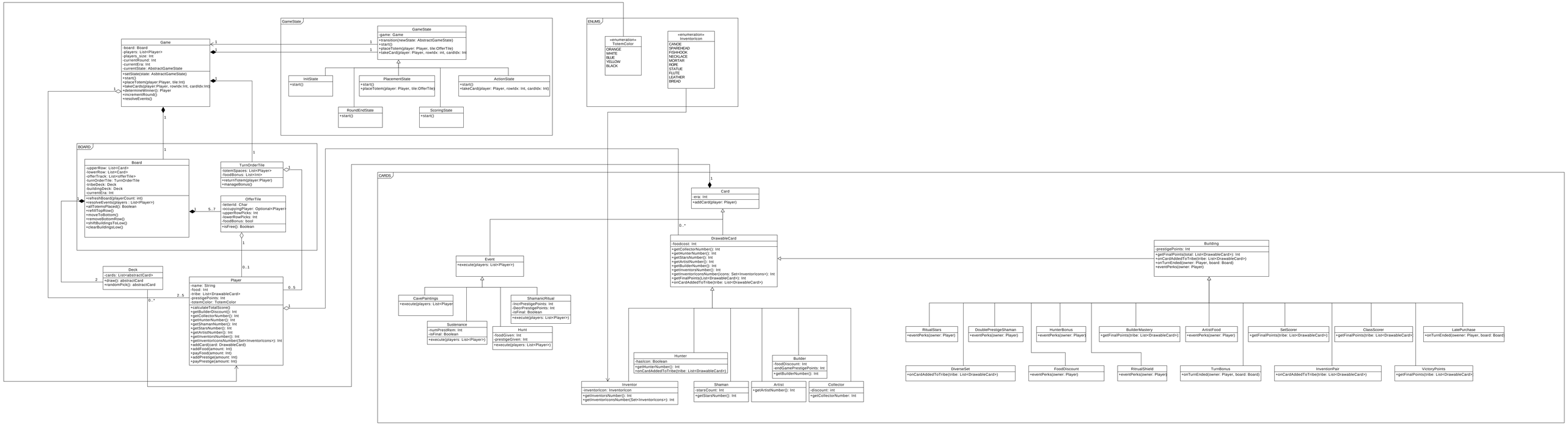
GameState
Game
-board: Board
-players: List<Player>
-players_size: Int
-currentRound: Int
-currentEra: Int
-currentState: AbstractGameState
+setState(state: AbstractGameState)
+start()
+placeTotem(player:Player, tile:Int)
+takeCards(player:Player, rowIdx:Int, cardIdx:Int)
+determineWinner(): Player
+incrementRound()
+resolveEvents()
GameState
-game: Game
+transition(newState: AbstractGameState)
+start()
+placeTotem(player: Player, tile:OfferTile)
+takeCard(player: Player, rowIdx: int, cardIdx: Int)
InitState
+start()
PlacementState
+start()
+placeTotem(player: Player, tile:OfferTile)
ActionState
+start()
+takeCard(player: Player, rowIdx: Int, cardIdx: Int)
RoundEndState
+start()
ScoringState
+start()
ENUMS
«enumeration»
TotemColor
ORANGE
WHITE
BLUE
YELLOW
BLACK
«enumeration»
InventorIcon
CANOE
SPEARHEAD
FISHHOOK
NECKLACE
MORTAR
ROPE
STATUE
FLUTE
LEATHER
BREAD
BOARD
Board
-upperRow: List<Card>
-lowerRow: List<Card>
-offerTrack: List<offerTile>
-turnOrderTile: TurnOrderTile
-tribeDeck: Deck
-buildingDeck: Deck
-currentEra: Int
+refreshBoard(playerCount: int)
+resolveEvents(players : List<Player>)
+allTotemsPlaced(): Boolean
+refillTopRow()
+moveToBottom()
+removeBottomRow()
+shiftBuildingsToLow()
+clearBuildingsLow()
TurnOrderTile
-totemSpaces: List<Player>
-foodBonus: List<Int>
+returnTotem(player:Player)
+manageBonus()
OfferTile
-letterId: Char
-occupyingPlayer: Optional<Player>
-upperRowPicks: Int
-lowerRowPicks: Int
-foodBonus: bool
+isFreed(): Boolean
Deck
-cards: List<abstractCard>
+draw(): abstractCard
+randomPick(): abstractCard
Player
-name: String
-food: Int
-tribe: List<DrawableCard>
-prestigePoints: Int
-totemColor: TotemColor
+calculateTotalScore()
+getBuilderDiscount(): Int
+getCollectorNumber(): Int
+getHunterNumber(): Int
+getShamanNumber(): Int
+getStarsNumber(): Int
+getArtistNumber(): Int
+getInventorsNumber(): Int
+getInventorIconsNumber(Set<InventorIcons>): Int
+addCard(card: DrawableCard)
+addFood(amount: Int)
+payFood(amount: Int)
+addPrestige(amount: Int)
+payPrestige(amount: Int)
CARDS
Card
-era: Int
+addCard(player: Player)
DrawableCard
-foodcost: Int
+getCollectorNumber(): Int
+getHunterNumber(): Int
+getStarsNumber(): Int
+getArtistNumber(): Int
+getBuilderNumber(): Int
+getInventorsNumber(): Int
+getInventorIconsNumber(icons: Set<InventorIcons>): Int
+getFinalPoints(List<DrawableCard>): Int
+onCardAddedToTribe(tribe: List<DrawableCard>)
Event
+execute(players: List<Player>)
CavePaintings
+execute(players: List<Player>)
ShamanicRitual
-IncrPrestigePoints: Int
-DecrPrestigePoints: Int
-isFinal: Boolean
+execute(players: List<Player>)
Sustenance
-numPrestRem: Int
-isFinal: Boolean
+execute(players: List<Player>)
Hunt
-foodGiven: Int
-prestigeGiven: Int
+execute(players: List<Player>)
Hunter
-hasIcon: Boolean
+getHunterNumber(): Int
+onCardAddedToTribe(tribe: List<DrawableCard>)
Builder
-foodDiscount: Int
-endGamePrestigePoints: Int
+getBuilderNumber(): Int
Inventor
-inventorIcon: InventorIcon
+getInventorsNumber(): Int
+getInventorIconsNumber(Set<InventorIcons>): Int
Shaman
-starsCount: Int
+getStarsNumber(): Int
Artist
+getArtistNumber(): Int
Collector
-discount: int
+getCollectorNumber: Int
Building
-prestigePoints: Int
+getFinalPoints(total: List<DrawableCard>): Int
+onCardAddedToTribe(tribe: List<DrawableCard>)
+onTurnEnded(owner: Player, board: Board)
+eventPerks(owner: Player)
RitualStars
+eventPerks(owner: Player)
DoublePrestigeShaman
+eventPerks(owner: Player)
HunterBonus
+eventPerks(owner: Player)
BuilderMastery
+getFinalPoints(tribe: List<DrawableCard>)
ArtistFood
+eventPerks(owner: Player)
SetScorer
+getFinalPoints(tribe: List<DrawableCard>)
ClassScorer
+getFinalPoints(tribe: List<DrawableCard>
LatePurchase
+onTurnEnded(owner: Player, board: Board)
DiverseSet
+onCardAddedToTribe(tribe: List<DrawableCard>)
FoodDiscount
+eventPerks(owner: Player)
RitualShield
+eventPerks(owner: Player)
TurnBonus
+onTurnEnded(owner: Player, board: Board)
InventionPair
+onCardAddedToTribe(tribe: List<DrawableCard>)
VictoryPoints
+getFinalPoints(tribe: List<DrawableCard>
1
0..*
2..5
5..7
0..1
0..5